

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS
FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA
ESCUELA PROFESIONAL DE INGENIERÍA DE TELECOMUNICACIONES



TÍTULO:

“Implementación Física, Síntesis RTL, Visualización en Displays y Ejecución de Comandos AT”

INTEGRANTES:

Chuzon Wickham, Renzo Felipe

Rivera Flores, Jorge David

Rodriguez Vasquez, Andrea Alexandra

INTRODUCCIÓN Y ARQUITECTURA DEL SISTEMA:

En este proyecto se utiliza la tarjeta de desarrollo Terasic DE10-Lite equipada con una FPGA Intel MAX 10 (modelo 10M50DAF484C7G).

Toda la lógica del sistema comparte el reloj maestro de 50 MHz (clk), ralentizando los procesos mecánicos (escaneo matricial y antirrebote) a 1 kHz mediante un Clock Enable.

El flujo arquitectónico se define en las siguientes etapas:

1. **Acondicionamiento y Escaneo:** Un generador de Tick estabiliza el escaneo de la Máquina de Estados (FSM) que multiplexa las filas del teclado en bajo (0). Un filtro digital de 20 ms ignora los rebotes mecánicos de las teclas.
2. **Registro Concurrente:** Las coordenadas son decodificadas a formato ASCII de 8 bits y guardadas secuencialmente en un banco de cuatro registros (reg0 a reg3).
3. **Visualización Ergonómica (Izquierda a Derecha):** Para una lectura natural del usuario, las salidas de memoria han sido entrelazadas de forma inversa hacia los decodificadores de los displays. El primer dato ingresado (reg0) se rutea al display situado más a la izquierda (HEX3), mientras el último (reg3) finaliza en la derecha (HEX0).
4. **Unidad Ejecutora Combinacional (Control de Dispositivos):** Una compuerta de comparación extendida analiza simultáneamente los 32 bits del búfer (reg0 a reg3). Si la contraseña introducida corresponde al comando AT A001, se enciende un actuador físico (LED). Si corresponde a A000, se desactiva.

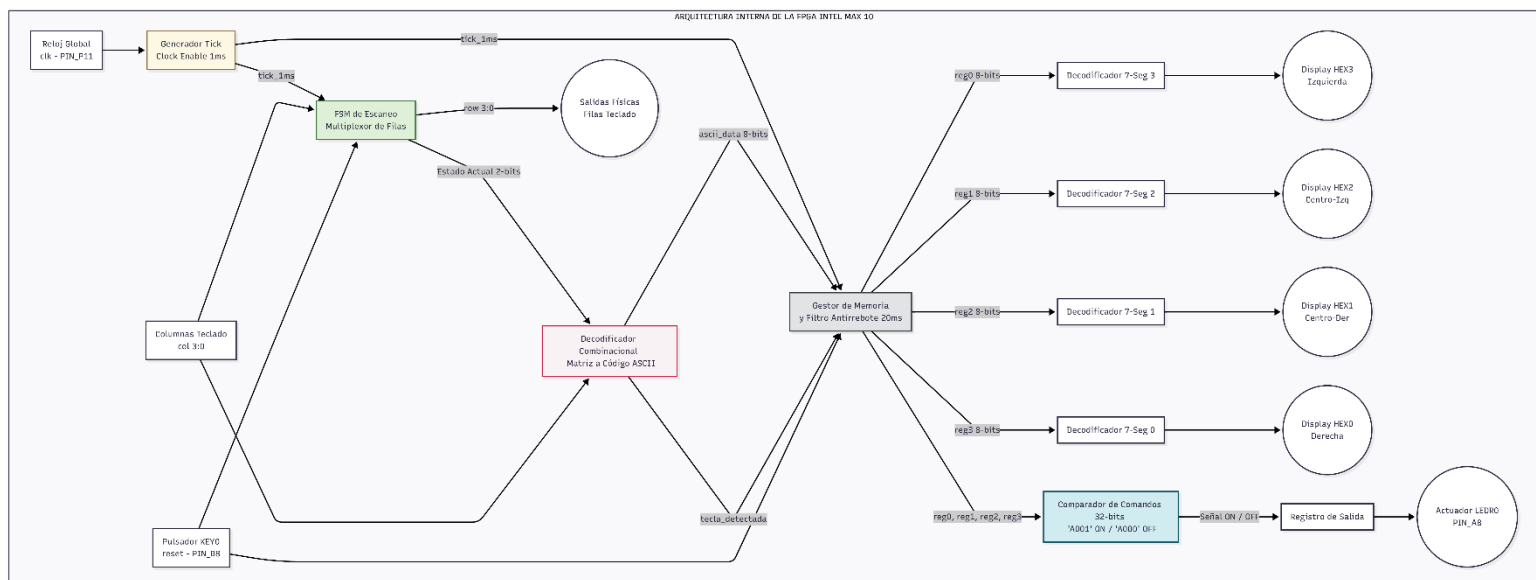


Figura 1. Diagrama de conexiones.

PINOUT:

El siguiente listado representa el mapa de conexiones exacto cargado en el Pin Planner de Quartus Prime operando bajo el voltaje por defecto de 2.5V.

NOMBRE DE LA SEÑAL (HDL)	TIPO	PIN FÍSICO FPGA	DESCRIPCIÓN Y CONEXIÓN EN HARDWARE
clk	Input	PIN_P11	Oscilador interno de 50 MHz
reset	Input	PIN_B8	Pulsador KEY0 integrado (Activo en Bajo)
col[0] a col[3]	Input	PIN_AB5 a AB8	Entradas de las Columnas del Teclado Matricial
row[0] a row[3]	Output	PIN_AB9 a AA12	Salidas de las Filas del Teclado Matricial
led_accion	Output	PIN_A8	Diodo LED Rojo LEDR0 (Verificación)
disp0[0:6]	Output	Pines fijos HEX0	Display HEX0 (Muestra Registro 3 - Derecha)
disp1[0:6]	Output	Pines fijos HEX1	Display HEX1 (Muestra Registro 2)
disp2[0:6]	Output	Pines fijos HEX2	Display HEX2 (Muestra Registro 1)
disp3[0:6]	Output	Pines fijos HEX3	Display HEX3 (Muestra Registro 0 - Izquierda)

Tabla 1. Asignación de pines.

FOTOGRAFÍAS CLARAS DEL PROTOTIPO ENSAMBLADO:

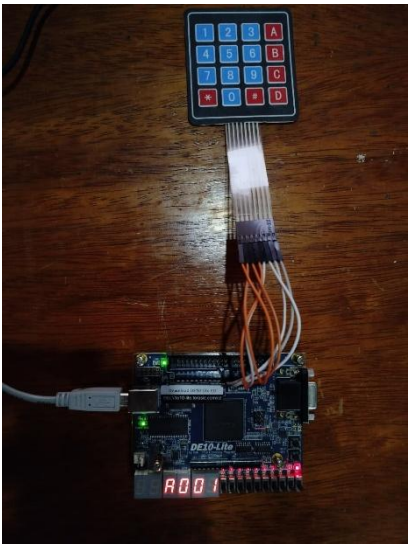
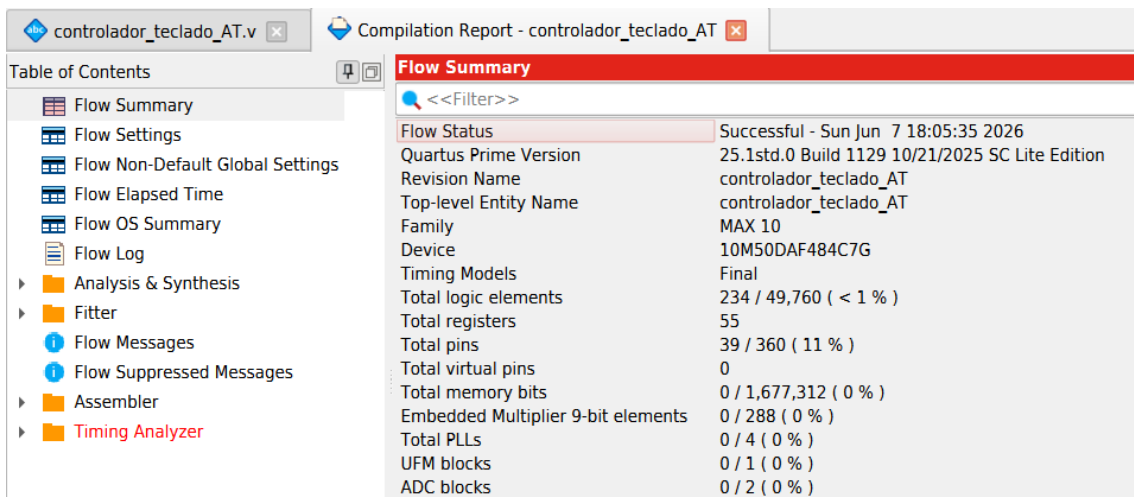


Figura 2. FPGA DE10-Lite conectado con el teclado matricial 4x4 mediante pines GPIO Arduino.

REPORTE DE RECURSOS OCUPADOS:

El resumen de utilización de recursos físicos de la FPGA es el siguiente:

- **Elementos Lógicos (LUTs) / Total logic elements:** 234 / 49,760 (< 1%). Usa una fracción mínima del chip.
- **Registros Dedicados (Flip-Flops) / Total registers:** 55. Estos 55 registros están distribuidos principalmente en los contadores del divisor de reloj (Tick de 1ms), la máquina de estados del escaneo de filas y los bancos de memoria para las teclas presionadas.
- **Pines de Entrada/Salida (I/O Pins):** 39 / 360 (11%). Es la suma exacta de nuestras conexiones físicas: el reloj maestro, reset, las 4 filas, las 4 columnas, el LED de comando y los 28 pines dedicados a los 4 displays de 7 segmentos.
- **Bits de Memoria RAM (Block RAM):** 0 / 1,677,312 (0%). La memoria de los 4 caracteres ingresados se implementó mediante registros distribuidos en lugar de bloques de memoria dedicados.



The screenshot shows the 'Flow Summary' window in Quartus. The left sidebar contains a 'Table of Contents' with items like Flow Summary, Flow Settings, Flow Non-Default Global Settings, Flow Elapsed Time, Flow OS Summary, Flow Log, Analysis & Synthesis, Fitter, Flow Messages, Flow Suppressed Messages, Assembler, and Timing Analyzer. The main area displays the following data:

Flow Summary	
<<Filter>>	
Flow Status	Successful - Sun Jun 7 18:05:35 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	controlador_teclado_AT
Top-level Entity Name	controlador_teclado_AT
Family	MAX 10
Device	10M50DAF484C7G
Timing Models	Final
Total logic elements	234 / 49,760 (< 1 %)
Total registers	55
Total pins	39 / 360 (11 %)
Total virtual pins	0
Total memory bits	0 / 1,677,312 (0 %)
Embedded Multiplier 9-bit elements	0 / 288 (0 %)
Total PLLs	0 / 4 (0 %)
UFM blocks	0 / 1 (0 %)
ADC blocks	0 / 2 (0 %)

Figura 3. Reporte de recursos utilizados extraído de Quartus.

CÓDIGO HDL:

A continuación, se presenta el código Verilog sintetizable que integra todas las funcionalidades utilizando la técnica profesional de Clock Enable para evitar problemas de sincronía en el hardware físico.

```
//  
MÓDULO: controlador_teclado_AT  
//  
  
module controlador_teclado_AT (  
    input wire      clk,          // Reloj maestro de la FPGA (50 MHz)
```

```

PIN_P11
    input wire      reset,          // Reset (KEY0 activo en bajo) PIN_B8
    input wire [3:0] col,          // Entradas: Columnas del teclado (Pull-
up)
    output reg [3:0] row,          // Salidas: Filas del teclado
    output reg      led_accion,    // LED físico en la placa (LEDR0)
    output wire [6:0] disp0,       // Display HEX0
    output wire [6:0] disp1,       // Display HEX1
    output wire [6:0] disp2,       // Display HEX2
    output wire [6:0] disp3       // Display HEX3
);

// 1. GENERADOR DE TICK (CLOCK ENABLE): 1 pulso cada 1 milisegundo
// Esta es la forma profesional de "ralentizar" el circuito en FPGA

reg [15:0] cont_1ms;
wire      tick_1ms = (cont_1ms == 16'd49_999); // Se hace 1 cada
50,000 ciclos

always @(posedge clk or negedge reset) begin
    if (~reset) begin
        cont_1ms <= 16'd0;
    end else begin
        if (tick_1ms) cont_1ms <= 16'd0;
        else          cont_1ms <= cont_1ms + 1'b1;
    end
end

// 2. MÁQUINA DE ESTADOS (FSM) PARA ESCANEO DE FILAS

localparam S0_FILA0 = 2'b00;
localparam S1_FILA1 = 2'b01;
localparam S2_FILA2 = 2'b10;
localparam S3_FILA3 = 2'b11;

reg [1:0] estado_actual, estado_siguiete;

// Usamos el reloj de 50MHz, pero SOLO avanzamos cuando hay tick_1ms
always @(posedge clk) begin
    if (~reset) begin
        estado_actual <= S0_FILA0;
    end else if (tick_1ms) begin // <--- CLOCK ENABLE AQUÍ
        estado_actual <= estado_siguiete;
    end
end

always @(*) begin
    case (estado_actual)
        S0_FILA0: begin row = 4'b1110; estado_siguiete = S1_FILA1; end
        S1_FILA1: begin row = 4'b1101; estado_siguiete = S2_FILA2; end
        S2_FILA2: begin row = 4'b1011; estado_siguiete = S3_FILA3; end
        S3_FILA3: begin row = 4'b0111; estado_siguiete = S0_FILA0; end
    endcase
end

```

```

        default: begin row = 4'b1111; estado_siguiente = S0_FILA0; end
    endcase
end

// 3. DECODIFICADOR COMBINACIONAL: MATRIZ -> ASCII

reg [7:0] ascii_data;
reg      tecla_detectada;

always @(*) begin
    ascii_data = 8'h00;
    tecla_detectada = 1'b0;
    case ({estado_actual, col})
        {S0_FILA0, 4'b1110}: begin ascii_data = 8'h31; tecla_detectada
= 1'b1; end // '1'
        {S0_FILA0, 4'b1101}: begin ascii_data = 8'h32; tecla_detectada
= 1'b1; end // '2'
        {S0_FILA0, 4'b1011}: begin ascii_data = 8'h33; tecla_detectada
= 1'b1; end // '3'
        {S0_FILA0, 4'b0111}: begin ascii_data = 8'h41; tecla_detectada
= 1'b1; end // 'A'
        {S1_FILA1, 4'b1110}: begin ascii_data = 8'h34; tecla_detectada
= 1'b1; end // '4'
        {S1_FILA1, 4'b1101}: begin ascii_data = 8'h35; tecla_detectada
= 1'b1; end // '5'
        {S1_FILA1, 4'b1011}: begin ascii_data = 8'h36; tecla_detectada
= 1'b1; end // '6'
        {S1_FILA1, 4'b0111}: begin ascii_data = 8'h42; tecla_detectada
= 1'b1; end // 'B'
        {S2_FILA2, 4'b1110}: begin ascii_data = 8'h37; tecla_detectada
= 1'b1; end // '7'
        {S2_FILA2, 4'b1101}: begin ascii_data = 8'h38; tecla_detectada
= 1'b1; end // '8'
        {S2_FILA2, 4'b1011}: begin ascii_data = 8'h39; tecla_detectada
= 1'b1; end // '9'
        {S2_FILA2, 4'b0111}: begin ascii_data = 8'h43; tecla_detectada
= 1'b1; end // 'C'
        {S3_FILA3, 4'b1110}: begin ascii_data = 8'h2A; tecla_detectada
= 1'b1; end // '*'
        {S3_FILA3, 4'b1101}: begin ascii_data = 8'h30; tecla_detectada
= 1'b1; end // '0'
        {S3_FILA3, 4'b1011}: begin ascii_data = 8'h23; tecla_detectada
= 1'b1; end // '#'
        {S3_FILA3, 4'b0111}: begin ascii_data = 8'h44; tecla_detectada
= 1'b1; end // 'D'
        default: begin ascii_data = 8'h00; tecla_detectada = 1'b0; end
    endcase
end

// 4. GESTIÓN DE MEMORIA Y FILTRO ANTIRREBOTE (20ms)

reg [7:0] reg0, reg1, reg2, reg3;

```

```

reg      tecla_bloqueada;
reg [4:0] temporizador;
reg [1:0] ptr_direccion;

wire pulso_escritura = tecla_detectada & ~tecla_bloqueada;

// Reloj 50MHz, pero la lógica avanza con el tick_1ms
always @(posedge clk) begin
    if (~reset) begin
        reg0 <= 8'h00; reg1 <= 8'h00;
        reg2 <= 8'h00; reg3 <= 8'h00;
        ptr_direccion <= 2'b00;
        tecla_bloqueada <= 1'b0;
        temporizador <= 5'd0;
    end else if (tick_1ms) begin // <--- CLOCK ENABLE AQUÍ

        if (tecla_detectada) begin
            tecla_bloqueada <= 1'b1;
            temporizador <= 5'd0;
        end else begin
            if (temporizador < 5'd20) begin
                temporizador <= temporizador + 1'b1;
            end else begin
                tecla_bloqueada <= 1'b0;
            end
        end
    end

    if (pulso_escritura) begin
        case (ptr_direccion)
            2'b00: reg0 <= ascii_data;
            2'b01: reg1 <= ascii_data;
            2'b10: reg2 <= ascii_data;
            2'b11: reg3 <= ascii_data;
        endcase
        ptr_direccion <= ptr_direccion + 1'b1;
    end
end

// 5. EJECUCIÓN DE COMANDO REAL (Contraseña de 4 caracteres)

always @(posedge clk) begin
    if (~reset) begin
        led_accion <= 1'b0;
    end else begin
        // Comando ON: "A001" (A, 0, 0, 1)
        if (reg0 == 8'h41 && reg1 == 8'h30 && reg2 == 8'h30 && reg3 ==
8'h31)
            led_accion <= 1'b1;

        // Comando OFF: "A000" (A, 0, 0, 0)
        else if (reg0 == 8'h41 && reg1 == 8'h30 && reg2 == 8'h30 &&

```

```

reg3 == 8'h30)
        led_accion <= 1'b0;
    end
end

// 6. DECODIFICADORES DE PANTALLA (Corregido de Izquierda a Derecha)

// Ahora reg0 (1ra tecla) va a disp3 (Pantalla de la izquierda)
decodificador_7seg u3 (.ascii(reg0), .seg(dis3));
decodificador_7seg u2 (.ascii(reg1), .seg(dis2));
decodificador_7seg u1 (.ascii(reg2), .seg(dis1));
decodificador_7seg u0 (.ascii(reg3), .seg(dis0));

endmodule

// SUBMÓDULO: DECODIFICADOR ASCII -> 7 SEGMENTOS

module decodificador_7seg (
    input  wire [7:0] ascii,
    output reg  [6:0] seg
);
    always @(*) begin
        case (ascii)
            8'h30: seg = 7'b1000000; // '0'
            8'h31: seg = 7'b1111001; // '1'
            8'h32: seg = 7'b0100100; // '2'
            8'h33: seg = 7'b0110000; // '3'
            8'h34: seg = 7'b0011001; // '4'
            8'h35: seg = 7'b0010010; // '5'
            8'h36: seg = 7'b0000010; // '6'
            8'h37: seg = 7'b1111000; // '7'
            8'h38: seg = 7'b0000000; // '8'
            8'h39: seg = 7'b0011000; // '9'
            8'h41: seg = 7'b0001000; // 'A'
            8'h42: seg = 7'b0000011; // 'B'
            8'h43: seg = 7'b1000110; // 'C'
            8'h44: seg = 7'b0100001; // 'D'
            8'h2A: seg = 7'b0011100; // '*'
            8'h23: seg = 7'b1001000; // '#'
            8'h00: seg = 7'b1111111; // Apagado
            default: seg = 7'b1111111;
        endcase
    end
endmodule

```

ANÁLISIS TEÓRICO DE LAS MÉTRICAS:

En la simulación se evaluó el tiempo de multiplexación del controlador del teclado. Como se observa en la gráfica, el sistema inicia en estado de reposo (col = 1111). Al simular la pulsación de una tecla (col = 1110), la máquina de estados abandona el estado de espera y comienza la secuencia de escaneo en la salida row, barriendo los valores 1110, 1101,

1011 y 0111.

Comparación de la métrica:

- **Valor Simulado:** Al medir con los cursores el tiempo entre cada cambio de fila en la gráfica, se obtiene un delta constante.
- **Valor Físico:** En la implementación física sobre la FPGA, el divisor de frecuencia original establece esta métrica en exactamente 1 milisegundo por fila. Esta es la métrica que se contrastará cuantitativamente en el laboratorio utilizando las puntas del osciloscopio sobre los pines de salida row, esperando medir un período de 4 ms por ciclo completo.

